

Mastering User Authentication in Rails with Devise and OAuth

Revision Date: 9/18/25

Original Author: Yihao Xie, TA, CSCE 431 - Spring 2021, Fall 2021

Revised Primer: Pauline Wade & Sahithi Ravipati, CSCE 431 – Fall, 2023 Spring 2023

Test App Code: Vishal Adepu, CSCE 431 – Spring 2023

Revised Primer: Riddhi Ghate CSCE 431 – Fall, 2025

Objectives

Rails does not come with a built-in authentication mechanism. However, developers can use several useful tools that provide all necessary authentication functions.

In this primer, we will discuss how to implement authentication using 3rd party authentication using Google OAuth with OmniAuth & Devise gems.

Adapted from

- Tutorial: Written by Adam Langsner; Nov 11, 2023
<https://medium.com/@adamlangsner/google-oauth-rails-5-using-devise-and-omniauth-1b7fa5f72c8e>
- Code source & example rails app
tutorial: https://github.com/adamlangsner/google_oauth_devise.git
If you want to copy the code source, modify Gemfile accordingly and supply the client ID and client Secret environment variables.
- Test Code: https://github.com/paulinewade/blog_secure_fa25.git

Other References:

- Describes many common security problems in web applications and how to avoid them with Rails. <https://guides.rubyonrails.org/security.html>

Helpful links for issues encountered while integrating

<https://stackoverflow.com/questions/66086606/could-not-authenticate-you-from-googleoauth2-because-authenticity-error>

<https://stackoverflow.com/questions/66960490/argumenterror-wrong-number-of-arguments-in-google-oauth-callback>

Contents

Overview of Incorporating OAuth	3
<i>Installing Devise and OmniAuth</i>	3
<i>Devise Admin Model and Migration</i>	4
Admin Model (models/admin.rb).....	4
Admin Migration	4
<i>Devise Routes</i>	5
<i>Login Page and Admin Dashboard Page</i>	6
Creating the Controllers	6
Creating the Views	7
Deploy to Localhost.....	10
<i>Configuring OAuth in Google Cloud Console</i>	11
<i>Handling OAuth Callback Request</i>	19
<i>Log in</i>	21
<i>Creating the Heroku app</i>	22

Overview of Incorporating OAuth

The setup is not conceptually difficult but has many steps. The step-by-step guide is for a barebones implementation, in the context of an Admin model but it works just as well for end users or other identities.

Installing Devise and OmniAuth

Add the following gems to your Gemfile.

```
gem 'devise', '~>4.9'
gem "omniauth", "~> 2.1"
gem "omniauth-rails_csrf_protection", "~> 1.0"
gem "omniauth-google-oauth2", "~> 1.1"
```

Run `bundle install`

Ignore this error if you get it, it will be fixed in the next steps:

```
[DEVISE] Please review the [changelog] and [upgrade guide] for more info on Hotwire / Turbo integration.

[changelog] https://github.com/heartcombo/devise/blob/main/CHANGELOG.md
[upgrade guide] https://github.com/heartcombo/devise/wiki/How-To:-Upgrade-to-Devise-4.9.0-%5BHotwire-Turbo-integration%5D
Post-install message from oauth2:

----- oauth2 v2.0.17 -----

(minor) ⚠️BREAKING CHANGES ⚠️when upgrading from < v2
• Summary of breaking changes: https://gitlab.com/ruby-oauth/oauth2#what-is-new-for-v20
• Changes in this patch: https://gitlab.com/ruby-oauth/oauth2/-/blob/v2.0.17/CHANGELOG.md#2015-2025-09-08

News:
1. New documentation website, including for OAuth 2.1 and OIDC: https://oauth2.galtzo.com
2. New official Discord for discussion and support: https://discord.gg/3qme4XHNKN
3. New org name "ruby-oauth" on Open Source Collective, GitHub, GitLab, Codeberg (update git remotes!)
4. Non-commercial support for the 2.x series will end by April, 2026. Please make a plan to upgrade to the next version prior to that date.
Support will be dropped for Ruby 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 3.0, 3.1 and any other Ruby versions which will also have reached EOL by then.
5. Gem releases are cryptographically signed with a 20-year cert; SHA-256 & SHA-512 checksums by stone_checksums.
6. Please consider supporting this project:
  • https://opencollective.com/ruby-oauth (new!)
  • https://liberapay.com/pboling
  • https://github.com/sponsors/pboling
  • https://www.paypal.com/paypalme/peterboling
  • https://ko-fi.com/pboling
  • https://www.buymeacoffee.com/pboling
  • https://tidelift.com/funding/github/rubygems/oauth
  • Hire me - I can build anything
  • Report issues, and star the project

Thanks, @pboling / @galtzo
```

Now run

```
rails generate devise:install
```

In `config/initializers`, this creates `devise.rb` and in `config/locales` a `devise.en.yml` translation file. It also modifies your routes file. When run, it will print four additional setup instructions. You can ignore these for now, we will do them as we go along.

Open the `devise.rb` file and add the following anywhere inside the setup block to configure OmniAuth:

```
config.omniauth :google_oauth2, ENV['GOOGLE_OAUTH_CLIENT_ID'],  
ENV['GOOGLE_OAUTH_CLIENT_SECRET']
```

Devise Admin Model and Migration

Now that we have Devise and OmniAuth installed, this section provides steps for creating the Admin ActiveRecord model and migration with Devise.

Generate the Admin model, by running

```
rails generate devise Admin
```

This creates a model and a migration that we will need to edit.

Admin Model (models/admin.rb)

Other devise modules in the generated model are not needed, only OmniAuth. Simplify as follows:

```
class Admin < ApplicationRecord
  devise :omniauthable, omniauth_providers: [:google_oauth2]
end
```

Admin Migration

Other auto-generated code in the migration is not needed. Add some new columns to persist information we get from Google. In db/migrate, add the following in the devise_create_admins migration file

```
class DeviseCreateAdmins < ActiveRecord::Migration[8.0]
  def change
    create_table :admins do |t|
      t.string :email, null: false
      t.string :full_name
      t.string :uid
      t.string :avatar_url

      t.timestamps null: false
    end

    add_index :admins, :email, unique: true
  end
end
```

Run

```
rails db:migrate
```

to generate the `admins` table.

Devise Routes

Devise added a `devise_for :admins` line of code to the routes file. It creates routes not needed. Replace with the bare minimum routes needed for OmniAuth, specified below:

```
Rails.application.routes.draw do
  root to: 'dashboards#show'

  devise_for :admins, controllers: { omniauth_callbacks:
'admins/omniauth_callbacks' }
  devise_scope :admin do
    get 'admins/sign_in', to: 'admins/sessions#new', as: :new_admin_session
    get 'admins/sign_out', to: 'admins/sessions#destroy',
as: :destroy_admin_session
  end
end
```

The following explains the code above:

- First we make the root path go to the dashboard.
- The next line generates two routes needed for OmniAuth: a `passthru` route that gets called when the user clicks login and a `callback` route that the browser redirects to after the user authenticates with Google.
- The last two routes are for the login page and the logout button. They are scoped to the `admins` namespace, so they do not conflict with login / logout routes for other potential identities like users.

Figure: Generated routes for Google OmniAuth Rails login

	Prefix	Verb	URI Pattern	Controller#Action
	root	GET	/	dashboards#show
admin_google_oauth2_omniauth_authorize		GET POST	/admins/auth/google_oauth2(.:format)	admins/omniauth_callbacks#passthru
admin_google_oauth2_omniauth_callback		GET POST	/admins/auth/google_oauth2/callback(.:format)	admins/omniauth_callbacks#google_oauth2
new_admin_session		GET	/admins/sign_in(.:format)	admins/sessions#new
destroy_admin_session		GET	/admins/sign_out(.:format)	admins/sessions#destroy

If you are implementing OAuth for an existing application, set root to your index page and set resources your routes file. For example, for your test app:

```

Rails.application.routes.draw do
  root 'books#index'

  devise_for :admins, controllers: { omniauth_callbacks:
'admins/omniauth_callbacks' }
  devise_scope :admin do
    get 'admins/sign_in', to: 'admins/sessions#new', as: :new_admin_session
    get 'admins/sign_out', to: 'admins/sessions#destroy',
as: :destroy_admin_session
  end
  resources :books
end

```

Login Page and Admin Dashboard Page

This section shows the steps for creating a basic login page and authenticated admin dashboard page.

Creating the Controllers

Create the dashboard controller in `app/controllers/dashboards_controller.rb`:

NOTE: You can skip this step if you already have an existing controller for your app

```

class DashboardsController < ApplicationController
  def show
  end
end

```

Don't skip this!

Next, the sessions controller in `app/controllers/admins/sessions_controller.rb`

```

class Admins::SessionsController < Devise::SessionsController
  def after_sign_out_path_for(_resource_or_scope)
    new_admin_session_path
  end

  def after_sign_in_path_for(resource_or_scope)
    stored_location_for(resource_or_scope) || root_path
  end
end

```

You do not need to implement `new` or `destroy` in the sessions controller because we inherit them from:

`Devise::SessionsController`

We just need to override the `after_sign_out_path_for` and `after_sign_in_path_for` methods.

You need to ensure that all controllers are authenticated by default, by adding code below to the ApplicationController in app/controllers

```
class ApplicationController < ActionController::Base
  before_action :authenticate_admin!
end
```

Devise generates the authenticate_admin!

This ensures that when an unauthenticated user tries to load any page they will get redirected to the login page.

Creating the Views

Create an authenticated dashboard view (with the code below) in views/dashboards/show.html.erb

```
<h1>You're logged in! Welcome to the Dashboard!</h1>
<%= link_to "Sign Out", destroy_admin_session_path %>
```

Note: In case you already have existing views, just add the code below in the corresponding index.html.erb

For example:

```
<p style="color: green"><%= notice %></p>

<h1>Books</h1>
<h1>This is the authenticated version 2</h1>

<h1>You're logged in! Welcome to the Dashboard!</h1>
<%= link_to "Sign Out", destroy_admin_session_path %>

<div id="books">
  <% @books.each do |book| %>
    <%= render book %>
    <p>
      <%= link_to "Show this book", book %>
    </p>
  <% end %>
</div>

<%= link_to "New book", new_book_path %>
```


And then for the login view, add code in

views/admins/sessions/new.html.erb :

```
<h2>You're Logged out! Log in to the Dashboard!!</h2>
```

```
<%= button_to admin_google_oauth2_omniauth_authorize_path, class: "btn btn-
danger", method: :post, data: {turbo: "false"} do %>
<i class="fab fa-google mobile-text"></i><span class = "mobile-
font">&nbsp;Google</span>
<% end %>
```

Devise uses the flash to communicate with the user, update the file below:

views/layouts/application.html.erb

```
<!DOCTYPE html>
<html>
  <head>
    <title>GoogleOauthDevise</title>
    <%= csrf_meta_tags %>
    <%= csp_meta_tag %>
    <%= stylesheet_link_tag      'application', media: 'all', 'data-turbolinks-
track': 'reload' %>
    <%= javascript_include_tag 'application', 'data-turbolinks-track':
'reload' %>
    <style>
      .flash__alert    { color: red; }
      .flash__notice  { color: blue; }
      .flash__success { color: green; }
    </style>
  </head>

  <body>
    <% if flash.any? %>
      <% flash.each do |key, value| -%>
        <section class="flash flash__<%= key %>"><%=
value.html_safe %></section>
      <% end -%>
    <% end %>
    <%= yield %>
  </body>
</html>
```

The code in the body loops through the flash and prints out whatever is inside using the key as part of the class name. The `<style>` tag give some color to different flash types to easily differentiate them.

In case you are getting javascript related errors, check that you have the following line in `app\assets\config\manifest.js`

```
//= link application.js
```

Alternative:

For Rails 6 only, replace in “`app\views\layouts\application.html.erb`”

```
<%= javascript_include_tag 'application', 'data-turbolinks-track':  
  'reload' %>
```

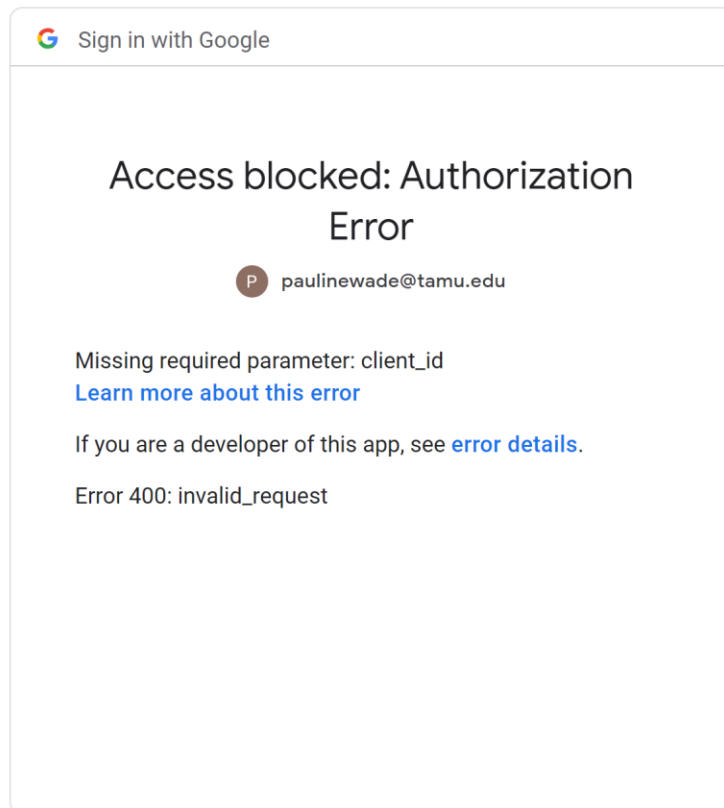
with

```
<%= javascript_pack_tag 'application', 'data-turbolinks-track':  
  'reload' %>
```

Deploy to Localhost

Start your rails server with `rails server --binding=0.0.0.0`, then go to `localhost:3000` and you will get redirected to `/admins/ sign_in` and you will see the following:

Figure: The Login Page



The link will not work yet because we have not configured Google Oauth, which is explained next.

Configuring OAuth in Google Cloud Console

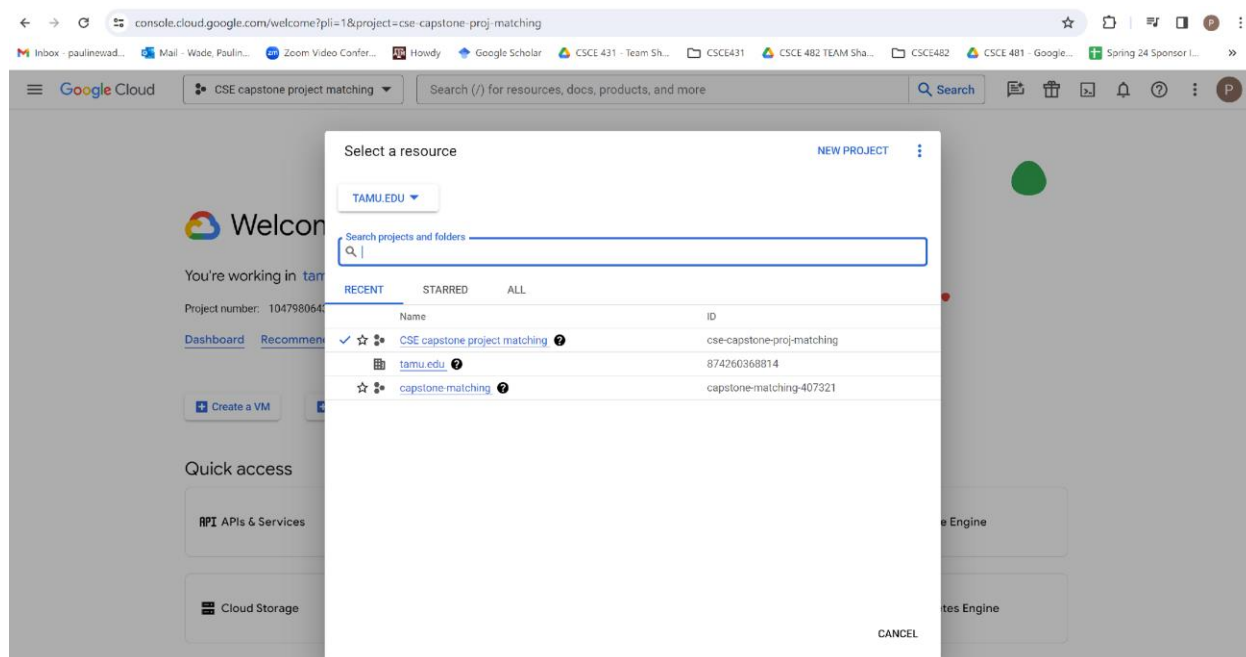
This section describes how to configure OAuth client ID in Google Cloud Console.

This part is a little confusing. If you are logged into multiple Google accounts it helps to **log out of all, except the one you want to use** for this app (e.g., log out of your personal Gmail if you intend to set this up for your work or school).

Go to <https://console.cloud.google.com>

Create a Project to house the OAuth Client ID that we will create.

Click “NEW PROJECT” to set up a new Project.



The name of the project is “Admin Dashboard” but name it anything you want. If you are doing this for your company or school (e.g., tamu.edu), you will want to create / select an organization to house the Project.

Search (/) for resources, docs, products, and more

Search

New Project

Project name *

Admin Dashboard

Project ID: admin-dashboard-415401. It cannot be changed later. [EDIT](#)

Organization *

tamu.edu

Select an organization to attach it to a project. This selection can't be changed later.

Location *

tamu.edu

[BROWSE](#)

Parent organization or folder

CREATE

CANCEL

Notifications

Create Project: Admin Dashboard

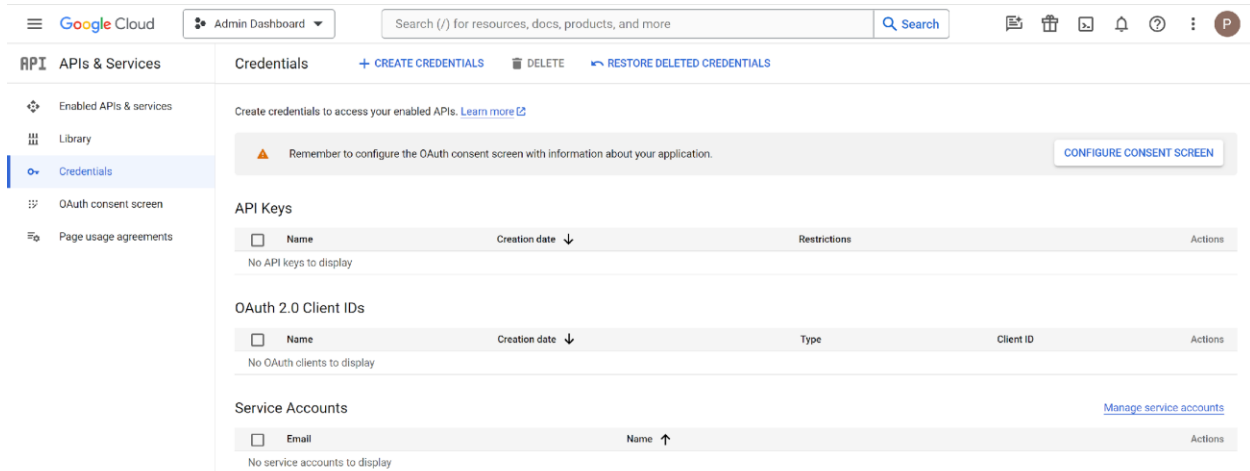
Just now

[SELECT PROJECT](#)

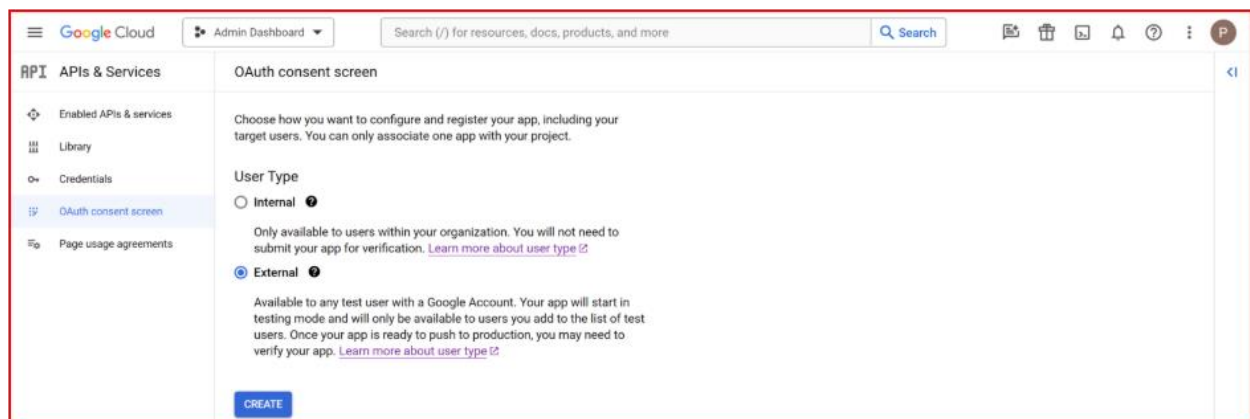
[SEE ALL ACTIVITIES](#)

Make sure your new project is selected.

Then, within “API & Services”, click “Credentials” on the left and click on the button “Configure Consent Screen.” Alternatively, select on the left “OAuth consent screen”



Select ‘External’ if asked



Go to the “Branding” tab and set the “Application Name”, “User Support email” and “Developer contact information.” Click Save and Continue. What you set here will be shown when the user sees Google’s account selection page.

Figure: Setting Application Name for OAuth consent screen

The screenshot shows the Google Cloud console interface for the 'google-oauth-test' project. The left sidebar contains navigation links: Overview, Branding (selected), Audience, Clients, Data Access, and Verification Center. The main content area is titled 'Branding' and is divided into two columns. The left column contains the 'App information' section, which includes fields for 'App name' (set to 'Admin Dashboard') and 'User support email' (set to 'riddhighate.07@tamu.edu'). Below this is the 'App logo' section with a 'Logo file to upload' field and a 'Browse' button. The 'App domain' section follows, with fields for 'Application home page', 'Application privacy policy link', and 'Application terms of service link'. The 'Authorized domains' section is also present, with a '+ Add domain' button. The right column shows the 'Verification status' as 'Verification is not required.' At the bottom of the main content area is the 'Developer contact information' section, with an 'Email addresses' field containing 'riddhighate.07@tamu.edu'. At the very bottom of the page, there are 'Save' and 'Discard changes' buttons.

Google Cloud google-oauth-test Search (/) for r

Google Auth Platform / Branding

Overview
Branding
Audience
Clients
Data Access
Verification Center

Branding

App information
This shows in the consent screen, and helps end users know who you are and contact you

App name *
Admin Dashboard
The name of the app asking for consent

User support email *
riddhighate.07@tamu.edu
For users to contact you with questions about their consent. [Learn more](#)

App logo
This is your logo. It helps people recognize your app and is displayed on the OAuth consent screen.
After you upload a logo, you will need to submit your app for verification unless the app is configured for internal use only or has a publishing status of "Testing".
[Learn more](#)

Logo file to upload [Browse](#)
Upload an image, not larger than 1MB on the consent screen that will help users recognize your app. Allowed image formats are JPG, PNG, and BMP. Logos should be square and 120px by 120px for the best results.

App domain
To protect you and your users, Google only allows apps using OAuth to use Authorized Domains. The following information will be shown to your users on the consent screen.

Application home page
Provide users a link to your home page

Application privacy policy link
Provide users a link to your public privacy policy

Application terms of service link
Provide users a link to your public terms of service

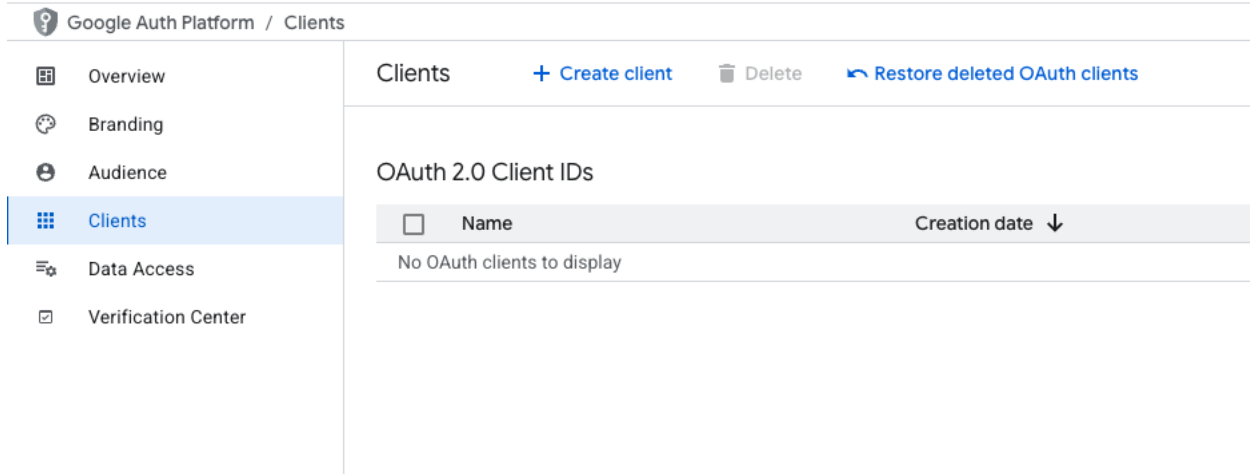
Authorized domains ⓘ
When a domain is used on the consent screen or in an OAuth client's configuration, it must be pre-registered here. If your app needs to go through verification, please go to the [Google Search Console](#) to check if your domains are authorized. [Learn more](#) about the authorized domain limit.

+ Add domain

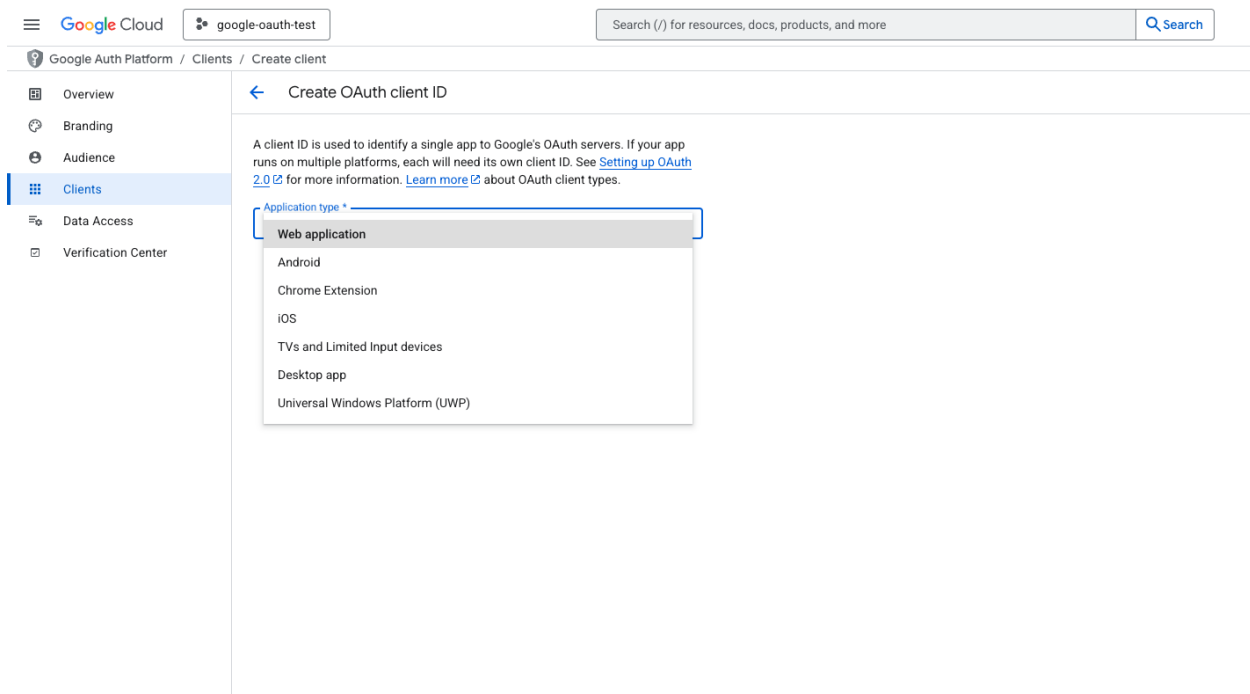
Developer contact information
Email addresses *
riddhighate.07@tamu.edu ⓘ
These email addresses are for Google to notify you about any changes to your project.

Save Discard changes

Go to the “Clients” tab on the left side and click on “Create Client” at the top.



Select “Web application” and provide a name. Name it “Development” to create one OAuth client ID per environment.



Under “Authorized redirect URIs”, add the path to the callback endpoint

`http://localhost:3000/admins/auth/google_oauth2/callback`

and press Create.

Google Cloud

google-oauth-test

Search (/) for resources, docs,

Google Auth Platform / Clients / Client: 563650489173-3nr1tc3jhdxfui1cnnqvi1fa107lq4hn.apps.googleusercontent.com

Overview

Branding

Audience

Clients

Data Access

Verification Center

Client ID for Web application

Delete

Name *

Development

The name of your OAuth 2.0 client. This name is only used to identify the client in the console and will not be shown to end users.

The domains of the URIs you add below will be automatically added to your [OAuth consent screen](#) as [authorized domains](#).

Authorized JavaScript origins

For use with requests from a browser

+ Add URI

Authorized redirect URIs

For use with requests from a web server

URIs 1 *

http://localhost:3000/admins/auth/google_oauth2/callback

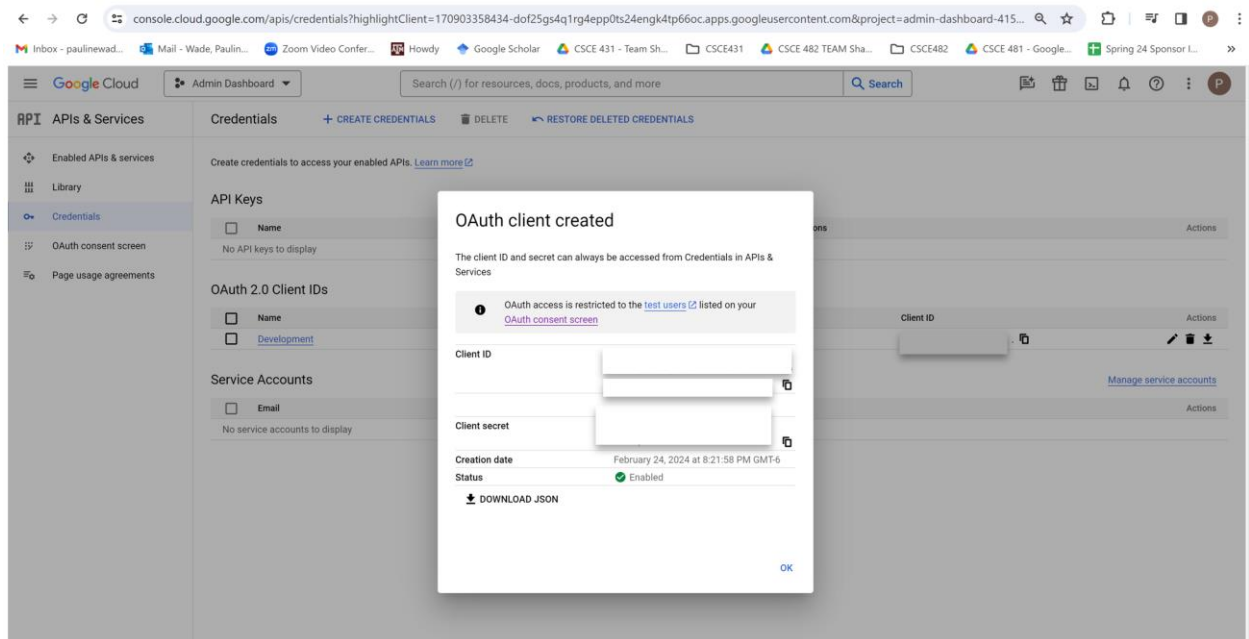
+ Add URI

Note: It may take 5 minutes to a few hours for settings to take effect

Save

Cancel

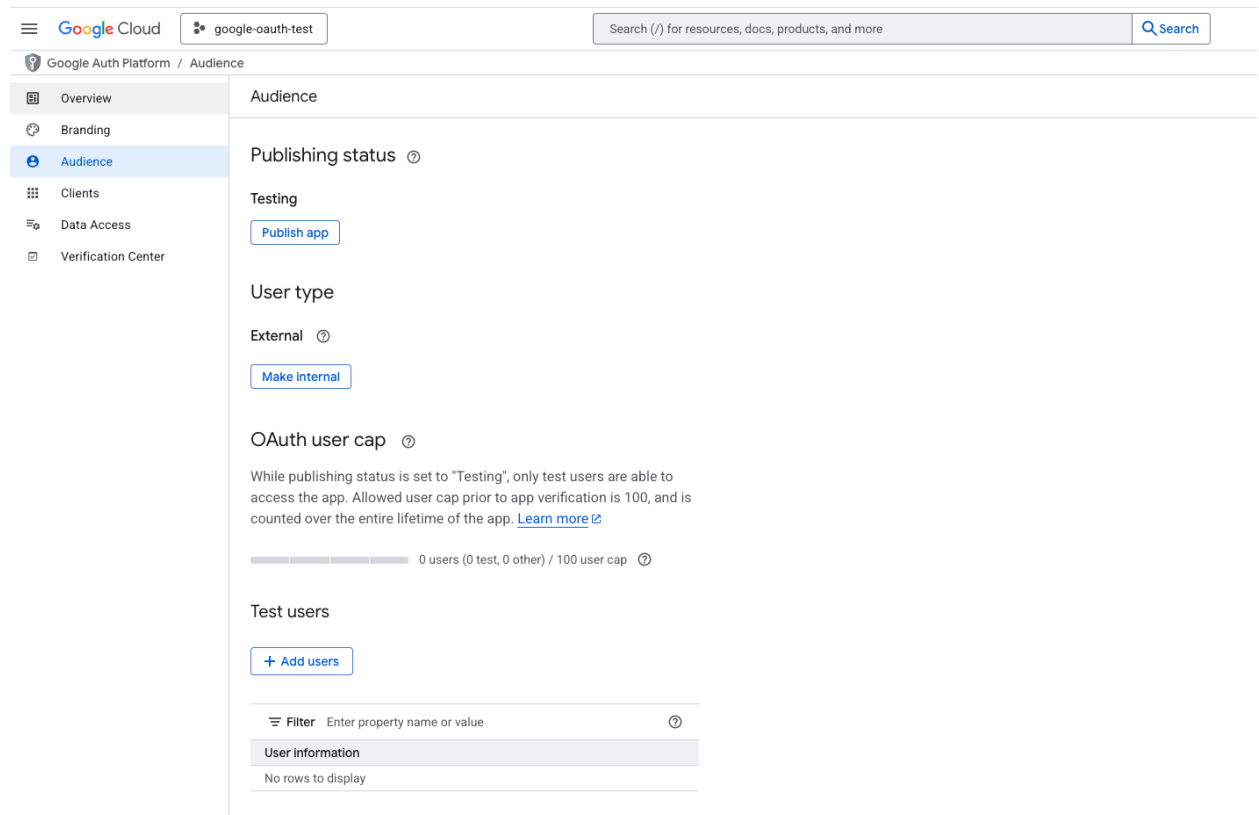
After you create the OAuth client ID you will see the client ID and client secret.



Copy these values to wherever you set your environment variables for local development. You can put at the bottom of the `development.rb` file located in `config/environments`:

```
ENV['GOOGLE_OAUTH_CLIENT_ID'] = '12345678901234-  
hkdshfdfsfdshfjdshfjdshfdssdkj.apps.googleusercontent.com'  
ENV['GOOGLE_OAUTH_CLIENT_SECRET'] = 'skjfgsdjkhdsjkhfhsjkhfhsjkhfjdsdk'
```

In the OAuth consent screen, go to “Audience” tab on the left and click on “Publish App” so that anyone can authenticate into your application. In the User type, select ‘External’.



The screenshot shows the Google Cloud console interface for the 'google-oauth-test' project. The left sidebar contains a navigation menu with 'Audience' selected. The main content area is titled 'Audience' and includes the following sections:

- Publishing status**: A button labeled 'Publish app'.
- User type**: A dropdown menu currently showing 'External', with a 'Make internal' button below it.
- OAuth user cap**: A progress bar indicating '0 users (0 test, 0 other) / 100 user cap'. A text note explains that while the status is 'Testing', only test users can access the app, and the cap is 100.
- Test users**: A button labeled '+ Add users'.
- Table**: A table with a filter bar and a header 'User information'. The table body is empty, showing 'No rows to display'.

Handling OAuth Callback Request

This section describes handling OAuth Callback to find or create an admin in a rails app. You will need two things: a controller and a new method on our `Admin` model.

Create the OmniAuth callbacks controller in

`controllers/admins/omniauth_callbacks_controller.rb` :

```
class Admins::OmniauthCallbacksController <
  Devise::OmniauthCallbacksController
  def google_oauth2
    admin = Admin.from_google(**from_google_params)

    if admin.present?
      sign_out_all_scopes
      flash[:success] = t 'devise.omniauth_callbacks.success', kind: 'Google'
      sign_in_and_redirect admin, event: :authentication
    else
      flash[:alert] = t 'devise.omniauth_callbacks.failure', kind: 'Google',
reason: "#{auth.info.email} is not authorized."
      redirect_to new_admin_session_path
    end
  end
  protected

  def after_omniauth_failure_path_for(_scope)
    new_admin_session_path
  end

  def after_sign_in_path_for(resource_or_scope)
    stored_location_for(resource_or_scope) || root_path
  end

  private

  def from_google_params
    @from_google_params ||= {
      uid: auth.uid,
      email: auth.info.email,
      full_name: auth.info.name,
      avatar_url: auth.info.image
    }
  end

  def auth
    @auth ||= request.env['omniauth.auth']
  end
end
```

The `google_oauth2` method is the action for the callback route. It is called after the

user selects their Google account. The request has information about the user that just authenticated. Check to see if an admin record can be found, and if so, sign in and redirect to the dashboard with a success flash.

Otherwise, sign out and go back to the login page with an alert flash.

Let us implement the `Admin.from_google` method `self.from_google`. Add the lines in bold to the `Admin` model:

```
class Admin < ApplicationRecord
  devise :omniauthable, omniauth_providers: [:google_oauth2]

  def self.from_google(email:, full_name:, uid:, avatar_url:)
    create_with(uid: uid, full_name: full_name, avatar_url:
avatar_url).find_or_create_by!(email: email)
  end
end
```

This method first checks that the user's email is of the correct domain. If it is, the code does a 'find' or 'create' for an admin record using the email address.

Verifying that the email's domain matches your company domain is a security measure to ensure only authorized individuals can log in to the dashboard.

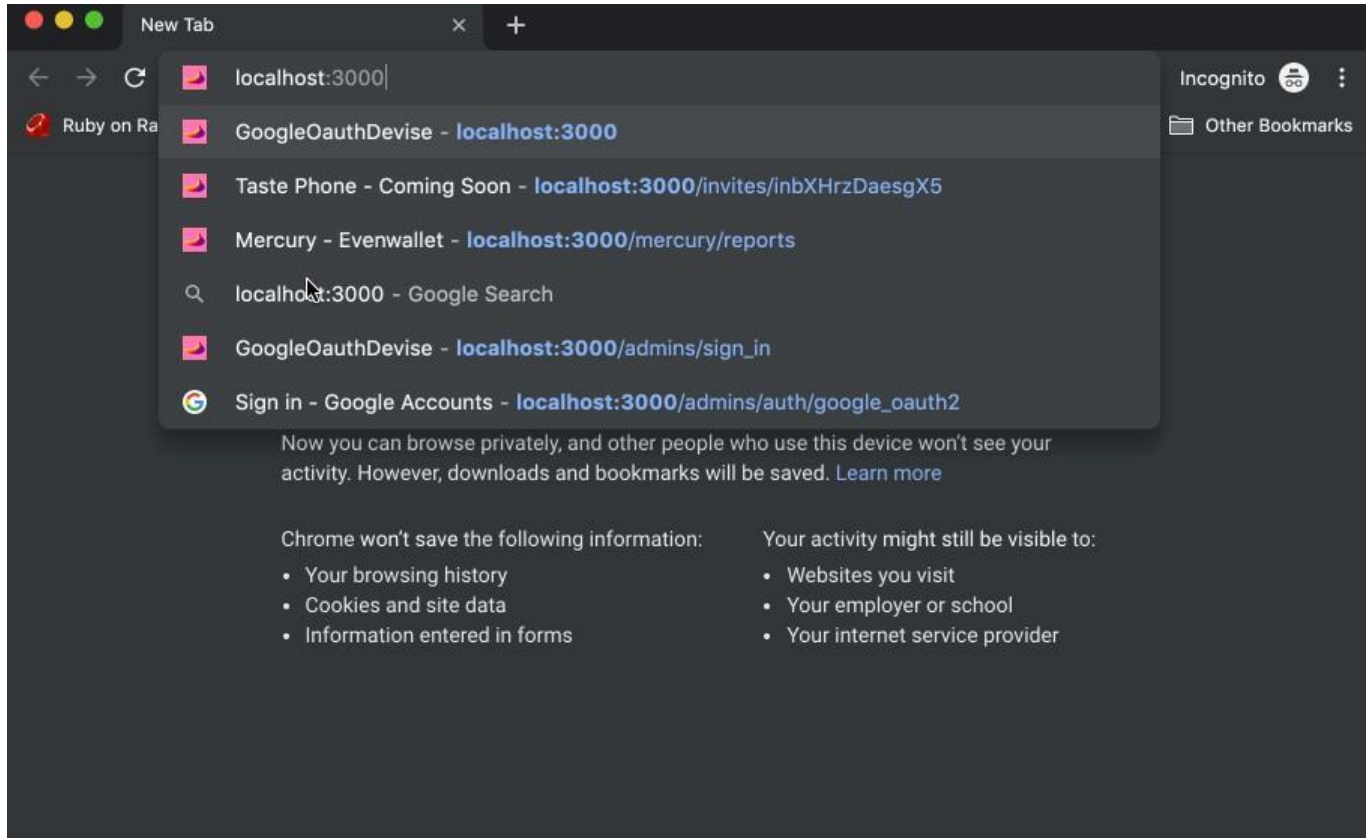
If this app is for end users you will probably want to remove that check.

There is also an opportunity for more security here. Doing a find or create of the admin record is great for a development or staging environment but might not be a good idea in production.

To add more security in production you could do just a 'find.' In the case of a company or organization, this means new employees or authorized individuals will not be able to log in automatically. Someone else in the company or organization will need to first create an admin record with the new employee's or member's email.

Log in

Now that we have all the code in place, let us log in:



Create the Heroku app, by doing the following:

- In Google Cloud Platform, add your *Heroku app url* to “Authorized Javascript Origins”.
- Then add the callback path to “Authorized redirect URIs.” For example:
https://yourapp.herokuapp.com/admins/auth/google_oauth2/callback

- Push

The screenshot shows the Google Cloud Admin Dashboard. On the left, the 'APIs & Services' menu is open, with 'Credentials' selected. The main panel is titled 'Client ID for Web application' and includes a 'DELETE' button. The 'Name' field is set to 'Development'. A note states: 'The domains of the URIs you add below will be automatically added to your OAuth consent screen as authorized domains.' Below this, the 'Authorized JavaScript origins' section is expanded, showing a list of URIs with one entry: 'https://cse-capstone-project-matching-3c6388be3453.herokuapp.com'. The 'Authorized redirect URIs' section is also expanded, showing one entry: 'https://yourapp.herokuapp.com/admins/auth/google_oauth2/callback'. At the bottom, there are 'SAVE' and 'CANCEL' buttons.

the app to Heroku